



Penjelasan Lengkap: `[build-system]` di `pyproject.toml` untuk Pemula

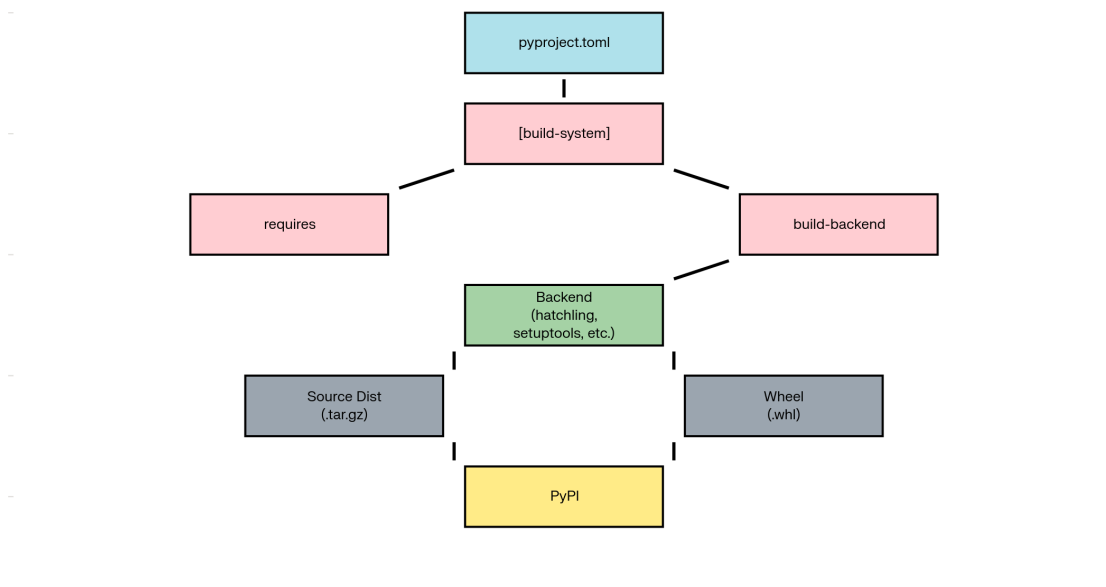
Sebagai pemula yang sudah menggunakan `uv`, sangat bagus bahwa Anda ingin memahami lebih dalam tentang `[build-system]`! Saya akan jelaskan dari dasar hingga advanced dengan cara yang mudah dipahami.

Apa Itu `[build-system]`?

`[build-system]` adalah **tabel konfigurasi** di `pyproject.toml` yang memberitahu Python **bagaimana cara mem-build (mengemas) project Anda**. Ini seperti **resep** yang menjelaskan:^[1]
^[2] ^[3] ^[4]

1. **Alat apa** yang dibutuhkan untuk build project
2. **Siapa** yang akan melakukan proses build-nya

Python Packaging Build Process



Alur Proses Build Python Package dari `pyproject.toml` hingga distribusi ke PyPI

Analogi Sederhana

Bayangkan Anda mau **membangun rumah**:

- `pyproject.toml` = **Blueprint rumah** (semua informasi tentang rumah)
- `[build-system]` = **Bagian blueprint yang menjelaskan kontraktor dan peralatan**
- `requires` = **Daftar peralatan yang dibutuhkan** (palu, gergaji, dll)
- `build-backend` = **Nama kontraktor** yang akan membangun rumah

Tanpa `[build-system]`, pip dan tools lain tidak tahu cara mengubah source code Anda menjadi package yang bisa di-install. [\[1\]](#) [\[2\]](#) [\[5\]](#)

Struktur `[build-system]`

```
[build-system]
requires = ["hatchling"]          # Tools yang dibutuhkan
build-backend = "hatchling.build" # Backend yang melakukan build
```

1. `requires` - Daftar Dependencies untuk Build

Kegunaan: Menentukan package Python apa saja yang **harus di-install** sebelum build process dimulai. [\[3\]](#) [\[4\]](#) [\[6\]](#)

Contoh:

```
[build-system]
requires = ["hatchling>=1.18.0"] # Butuh hatchling versi minimal 1.18.0
```

Atau untuk `setuptools`:

```
[build-system]
requires = ["setuptools>=61.0", "wheel"] # Butuh setuptools dan wheel
```

Kenapa penting?

- **Chicken-egg problem:** Dulu, untuk install package, Anda butuh `setuptools`. Tapi untuk tahu bahwa `setuptools` dibutuhkan, Anda harus baca `setup.py` yang butuh `setuptools` ter-install [\[2\]](#) [\[5\]](#) [\[7\]](#)
- **PEP 518** memecahkan masalah ini dengan `requires` - sekarang pip tahu **sebelum** mulai build, apa yang harus di-install dulu [\[5\]](#) [\[8\]](#) [\[2\]](#)

2. build-backend - Tool yang Melakukan Build

Kegunaan: Menentukan **Python module** mana yang akan dipanggil untuk melakukan proses build. [\[2\]](#) [\[3\]](#) [\[9\]](#)

Contoh berbagai backends:

```
# Hatchling (modern, PyPA backed)
build-backend = "hatchling.build"

# Setuptools (legacy/mature)
build-backend = "setuptools.build_meta"

# Flit (minimal/simple)
build-backend = "flit_core.buildapi"

# PDM
build-backend = "pdm.backend"

# Poetry
build-backend = "poetry.core.masonry.api"
```

Apa yang dilakukan backend?

Backend bertanggung jawab untuk: [\[6\]](#) [\[9\]](#) [\[2\]](#)

1. **Membaca metadata** project dari [project] table
2. **Mengumpulkan files** yang perlu di-include
3. **Membuat source distribution** (.tar.gz)
4. **Membuat wheel** (.whl) - format installable
5. **Meng-handle dependencies** yang dinamis

Kenapa Ada Banyak Backend? Perbandingan

Hatchling (Recommended untuk Pemula dan Modern Projects)

Contoh lengkap:

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "my_project"
version = "0.1.0"
```

Keunggulan untuk pemula: [\[10\]](#) [\[11\]](#) [\[12\]](#)

-  **Smart defaults:** Otomatis menggunakan .gitignore untuk tahu file mana yang di-include/exclude [\[10\]](#)

- ✓ **Tidak butuh konfigurasi tambahan:** "Just works" untuk 90% use cases
- ✓ **PyPA backed:** Didukung oleh Python Packaging Authority [\[11\]](#) [\[10\]](#)
- ✓ **Safe by default:** Tidak accidentally include files yang tidak seharusnya [\[10\]](#)

Setuptools (Legacy, tapi Masih Banyak Dipakai)

```
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "my_project"
version = "0.1.0"

[tool.setuptools.packages.find]
where = ["."]
```

Kapan pakai setuptools? [\[1\]](#) [\[13\]](#) [\[14\]](#)

- Project legacy yang sudah ada
- Butuh C/C++ extensions
- Backward compatibility penting
- Ecosystem yang sangat mature

Masalah setuptools: [\[10\]](#) [\[13\]](#)

- ✗ **Default buruk:** Include semua files kalau tidak dikonfigurasi
- ✗ **Butuh MANIFEST.in:** File terpisah untuk control apa yang di-include
- ✗ **Complex:** Banyak opsi membingungkan
- ✗ **Lambat:** Legacy code yang berat

Flit-core (Paling Simple)

```
[build-system]
requires = ["flit_core>=3.2"]
build-backend = "flit_core.buildapi"
```

Kapan pakai flit? [\[2\]](#) [\[13\]](#) [\[15\]](#)

- Pure Python package (no C extensions)
- Mau setup **secepat mungkin**
- Project sederhana tanpa customization

Proses Build: Apa yang Terjadi?

Mari kita trace apa yang terjadi ketika Anda run `pip install` . atau `uv build`:

Step 1: Baca `pyproject.toml`

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

Pip/uv membaca: "Ok, saya butuh install `hatchling` dulu". [\[4\]](#) [\[6\]](#)

Step 2: Install Build Dependencies

```
# Pip creates isolated environment
pip install hatchling # Install di temporary environment
```

Ini **isolated** - tidak mempengaruhi environment Anda. [\[2\]](#) [\[9\]](#)

Step 3: Call Build Backend

Pip/uv memanggil functions di `hatchling.build` module: [\[9\]](#)

```
# Internally, pip calls these hooks:
backend.build_wheel(wheel_directory)
backend.build_sdist(sdist_directory)
```

Step 4: Build Backend Bekerja

Hatchling:

1. Membaca `[project]` metadata dari `pyproject.toml`
2. Scan directory untuk Python files
3. Check `.gitignore` untuk exclude rules
4. Create `.tar.gz` (source distribution)
5. Create `.whl` (wheel/binary distribution)

Step 5: Output

```
dist/
├─ my_project-0.1.0.tar.gz # Source distribution
└─ my_project-0.1.0-py3-none-any.whl # Wheel
```

PEP 517 & PEP 518: Standar Yang Mengatur Ini

PEP 518 (2016)

Problem yang dipecahkan: "Chicken-egg" problem - butuh setuptools untuk install, tapi butuh install untuk punya setuptools. [\[2\]](#) [\[5\]](#) [\[7\]](#) [\[8\]](#)

Solusi: Introduce `[build-system]` table di `pyproject.toml`. [\[5\]](#) [\[8\]](#)

```
[build-system]
requires = ["setuptools"] # Declare dependencies BEFORE build
```

Sekarang pip tahu **sebelum mulai build**, harus install apa dulu. [\[8\]](#) [\[2\]](#)

PEP 517 (2017)

Problem yang dipecahkan: Semua orang terpaksa pakai setuptools, tidak ada alternatif. [\[2\]](#) [\[6\]](#) [\[9\]](#)

Solusi: Standardize **interface** yang harus diimplementasi oleh build backend: [\[9\]](#) [\[2\]](#)

- `build_wheel()` - Create wheel
- `build_sdist()` - Create source distribution
- `get_requires_for_build_wheel()` - Get dynamic dependencies

Dengan standar ini, siapa saja bisa buat build backend baru (hatchling, flit, dll) tanpa break pip/tools lain. [\[6\]](#) [\[2\]](#) [\[9\]](#)

Contoh Praktis: Membuat Project dengan UV

1. Initialize Project dengan UV

```
uv init my_agent --lib
cd my_agent
```

UV otomatis generate `pyproject.toml`:

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "my-agent"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.11"
dependencies = []
```

2. Tambah Dependencies

```
[project]
dependencies = [
    "openai>=1.0.0",
    "pydantic>=2.0.0",
]
```

3. Build Package

```
# UV akan:
# 1. Baca [build-system], install hatchling
# 2. Call hatchling.build
# 3. Generate dist/my_agent-0.1.0.whl

uv build
```

4. Install Package

```
# Install dari wheel yang di-build
pip install dist/my_agent-0.1.0-py3-none-any.whl

# Atau install langsung dari source (development mode)
pip install -e .
```

Kapan [build-system] Boleh Tidak Ada?

TIDAK PERNAH untuk modern projects! [\[1\]](#) [\[3\]](#) [\[12\]](#)

Jika tidak ada [build-system], pip akan fallback ke legacy behavior: [\[11\]](#) [\[9\]](#)

```
# Pip akan assume setuptools dan run:
python setup.py install # Legacy, deprecated!
```

Ini **buruk** karena: [\[5\]](#) [\[9\]](#)

- Tidak terisolasi
- Tidak reproducible
- Tidak support modern standards

Solusi: SELALU include [build-system]. [\[3\]](#) [\[4\]](#) [\[1\]](#)

Best Practices untuk Pemula

1. Gunakan Hatchling (Default UV)

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

Alasan: [\[10\]](#) [\[12\]](#) [\[16\]](#)

- Smart defaults, tidak butuh config tambahan
- PyPA backed (official)
- Fast dan modern
- Dipakai oleh UV by default [\[16\]](#)

2. Pin Version untuk Reproducibility

```
[build-system]
requires = ["hatchling>=1.18.0"] # Minimal version
```

3. Jangan Overthink Backend Choice

Untuk pemula, **ranking pilihan:** [\[13\]](#) [\[14\]](#) [\[15\]](#)

1. **Hatchling** - Modern, PyPA backed, great defaults
2. **Flit-core** - Jika mau yang super simple
3. **Setuptools** - Jika maintain legacy code
4. **PDM-backend** - Jika sudah pakai PDM ecosystem
5. **Poetry-core** - Jika sudah pakai Poetry

4. Understand Your Project Type

Pure Python (library, CLI tool, web app):

→ **Hatchling** atau **Flit-core**

C/C++ Extensions (scientific computing, performance):

→ **Setuptools** atau **Hatchling + plugins**

Legacy Project:

→ **Setuptools** (migration path)

Troubleshooting Common Issues

Issue 1: "No module named 'hatchling'"

Cause: Build backend tidak ter-install.^[4]

Solution:

```
# Build tools will auto-install, but manual:  
pip install hatchling
```

Issue 2: "Could not find build backend"

Cause: Typo di build-backend name.^[1]

Solution: Check nama module di dokumentasi backend:

```
# ✔ Correct  
build-backend = "hatchling.build"  
  
# ✘ Wrong  
build-backend = "hatchling" # Missing .build
```

Issue 3: Build include files yang tidak diinginkan

Cause: Backend defaults.^{[10] [12]}

Solution dengan Hatchling:

```
[tool.hatch.build.targets.wheel]  
exclude = [  
    "tests/",  
    "docs/",  
    ".github/",  
]
```

Solution dengan Setuptools:

Create MANIFEST.in:

```
exclude tests/*  
exclude docs/*
```

Kesimpulan

[build-system] adalah jantung dari Python packaging modern: ^[1] ^[2] ^[3]

1. Mendefinisikan dependencies untuk build (requires)
2. Mendefinisikan tool yang melakukan build (build-backend)
3. Menyelesaikan chicken-egg problem (PEP 518) ^[5] ^[8]
4. Memungkinkan alternatif selain setuptools (PEP 517) ^[2] ^[9]

Untuk pemula menggunakan UV: Anda sudah di track yang benar! UV default menggunakan Hatchling yang adalah pilihan terbaik untuk modern Python projects. ^[10] ^[11] ^[16]

Key takeaway:

- ✓ SELALU include [build-system] table
- ✓ Gunakan Hatchling untuk project baru (default UV)
- ✓ Tidak perlu overthink - defaults sudah bagus
- ✓ Focus pada code, let build system handle packaging

Sekarang Anda paham kenapa [build-system] ada dan bagaimana cara kerjanya! ☐



1. <https://realpython.com/python-pyproject-toml/>
2. <https://drivendata.co/blog/python-packaging-2023>
3. <https://packaging.python.org/en/latest/guides/writing-pyproject-toml/>
4. <https://packaging.python.org/en/latest/specifications/pyproject-toml/>
5. <https://pydevtools.com/handbook/explanation/what-is-pep-517/>
6. <https://labs.quansight.org/blog/pep-517-build-system-popularity>
7. https://www.reddit.com/r/Python/comments/io7e74/did_pep517_and_pep518_really_solve_the_setup/
8. <https://peps.python.org/pep-0518/>
9. <https://peps.python.org/pep-0517/>
10. <https://hatch.pypa.io/1.7/why/>
11. <https://github.com/sympy/sympy/issues/27116>
12. <https://www.pyopensci.org/python-package-guide/tutorials/pyproject-toml.html>
13. <https://www.pyopensci.org/python-package-guide/package-structure-code/python-package-build-tools.html>
14. <https://www.linkedin.com/pulse/comparing-python-build-backends-setuptools-hatchling-flit-sharma-cq2cf>
15. <https://packaging.python.org/guides/tool-recommendations/>
16. https://www.reddit.com/r/Python/comments/1gaz3tm/hatch_or_uv_for_a_new_project/
17. <https://stackoverflow.com/questions/62983756/what-is-pyproject-toml-file-for>
18. <https://xebia.com/blog/a-practical-guide-to-setuptools-and-pyproject-toml/>
19. <https://til.simonwillison.net/python/pyproject>

20. <https://stackoverflow.com/questions/71080546/what-is-the-preferred-way-to-develop-a-python-package-without-using-setup-py>